

GUION DE SUSTENTACION

Mini-JARVIS · Elena · Britany Macias

Pagina 1 = todo resumido. Si te pierdes, mira solo esta.

ANTES DE ENTRAR

- App abierta. Laboratorio ya ejecutado (tarda la primera vez).
- Internet y saldo comprobados. Volumen arriba.
- Abierto un solo archivo: exploration/transformer_lab.py

LAS 5 PIEZAS, EN UNA LINEA

Whisper oye · **Qwen** piensa · **Llama** piensa si el otro cae · **edge-tts** habla
BETO no hace nada de eso: se deja mirar por dentro.

LOS 5 BOTONES DE LA DEMO

- Laboratorio → ver los tokens
- Laboratorio → mapa de atencion
- Contador de memoria (bajo los deslizadores)
- Boton system prompt
- Temperatura: casi a cero, preguntar; al maximo, preguntar lo mismo

LAS 5 LINEAS DEL CODIGO

En VS Code: Ctrl+G, escribes el numero, Enter.

286 Tokenizacion

384 Embeddings y atencion

424 Softmax y temperatura

609 Positional encoding

808 Mapa de calor

SI ALGO FALLA

No te calles. Narra lo que pasa.

- Tarda → "es latencia de red, la peticion viaja a un servidor"
- No entiende → repite cerca del micro. Es el STT, no el modelo.
- Cae la red → perfecto: ensena el modo local
- Sale error → "esto es el manejo de errores, la app no se cae"
- No sabes algo → "eso no lo medi; lo que si compruebe es esto otro"

QUE DICES Y QUE SOLO RESPONDES

De las 7 paginas, hablas 2. El resto son respuestas esperando.

- Partes 1 y 2: las dices siempre. Son 6 minutos.
- Parte 5: las limitaciones las sacas tu, medio minuto.
- Cierre: siempre.
- Partes 3, 4 y el resto: SOLO si preguntan.

LAS CIFRAS

10 turnos de memoria · 10 herramientas · 189 pruebas · 86 commits

BETO: 12 capas x 12 cabezas = 144 mapas · 768 numeros por token · 512 posiciones

PARTE 1

La apertura

ESTO SE DICE SIEMPRE • un minuto

Lo primero que dices, sin que te pregunten. Un minuto.

DICES:

Mini-JARVIS es un asistente conversacional por voz en español. La asistente se llama Elena.

Hace tres cosas: oye, piensa y habla.

Oye con Whisper, que convierte audio en texto.

Piensa con un modelo de lenguaje grande, que lee ese texto y escribe la respuesta.

Y habla con edge-tts, que convierte la respuesta en voz.

Además trae un laboratorio aparte que abre el Transformer por dentro y enseña

que le pasa a una frase desde que entra hasta que sale.

No entrene ningún modelo. El enunciado dice que no hace falta y no es viable en un curso.

Lo que sí hice fue abrir uno para poder explicarlo.

Si te dan más tiempo, anade: y funciona también sin internet, que no lo pedían.

PARTE 2

La demostración en vivo

ESTO SE DICE SIEMPRE • unos cinco minutos

Cinco pasos. Cada uno: primero haces, luego hablas.

PASO 1 • Los tokens

HACER: Laboratorio → botón «ver los tokens».

DICES:

Un modelo de lenguaje no lee palabras como nosotros.

Antes de que el texto entre, un componente llamado tokenizador lo corta en pedazos y cambia cada pedazo por un número.

El modelo por dentro solo ve esa lista de números.

Aquí está mi frase cortada, con el número de cada pedazo.

Miren Mini-JARVIS: se parte en cuatro pedazos, porque no existe en el vocabulario del modelo.

Y batería y portátil se parten en tres cada una, porque llevan tilde.

Si salen símbolos raros tipo 'Ñña': no es un error. Este tokenizador trabaja sobre los bytes del texto, y una letra acentuada ocupa dos bytes.

PASO 2 • El mapa de atención

HACER: Laboratorio → botón «mapa de atención».

DICES:

Esto es el self-attention, que es el corazón del Transformer.

Cada fila es una palabra preguntando a quien mirar. Cada columna es a quien mira.

Lo brillante es donde pone su atención.

Fíjense en que la franja brillante no está en la diagonal: está una casilla a la izquierda.

Eso significa que cada palabra le presta casi toda su atención a la que va justo antes.

A eso se le llama una cabeza de token anterior.

Mi modelo tiene doce capas de doce cabezas: ciento cuarenta y cuatro mapas distintos

para esta misma frase, y cada uno aprendio a fijarse en un tipo de relacion diferente.

Elegi este porque el patron se ve claro.

Los tokens [CLS] y [SEP] los anade el modelo: marcan el principio y el final de la frase.

Por que importa: es lo que permite que una palabra signifique cosas distintas segun el contexto. Banco de rio no es banco de calle, y lo decide a quien atiende.

PASO 3 · La memoria

HACER: Senala el contador, debajo de los deslizadores: «Memoria: X de 10 turnos».

DICES:

Un modelo no recuerda nada entre una llamada y la siguiente.

La ilusion de que sigue el hilo se consigue reenviandole la conversacion entera cada vez.

Pero eso no puede crecer para siempre: hay un limite de contexto y ademas se paga por token.

Mi sistema guarda los ultimos diez turnos y descarta el mas antiguo, nunca el mas nuevo, porque lo que se acaba de decir es lo mas relevante.

Aqui se ve el contador. Corte por turnos y no por tokens a proposito:

por turnos es predecible y se puede enseñar en pantalla.

PASO 4 · El system prompt

HACER: Boton «system prompt».

DICES:

Esto es lo que recibe el modelo antes de cada frase mia, y es lo unico que define quien es Elena.

Le dice como se llama, que responde a su nombre, que es una inteligencia artificial y que puede equivocarse, y que escriba en frases cortas sin listas ni simbolos, porque sus respuestas se leen en voz alta y una vineta no se pronuncia.

PASO 5 · La temperatura

HACER: Baja la temperatura casi a cero, pregunta algo. Subela al maximo, pregunta lo mismo.

DICES:

La temperatura es cuanto riesgo corre el modelo al elegir cada palabra siguiente.

Cerca de cero elige siempre la mas probable: suena predecible, casi de manual.

Alta, se permite opciones raras: es mas creativo y tambien mas propenso a equivocarse.

El top_p va por otro camino: en vez de cambiar el riesgo de todas las opciones, se queda solo con las mas probables hasta acumular ese porcentaje.

Si preguntan por que el deslizador llega a 1.4 y no a 2: lo medi contra la API. A partir de 1.5 el modelo se atascaba unos cien segundos en dos de cada tres intentos.

PARTE 3

Las seis preguntas del enunciado

Solo si preguntan. No lo digas de corrido.

Cuatro ya las respondiste en la demostracion. Estas son las dos que faltan, mas el resumen de todas.

¿Que diferencia hay entre tu modelo y un modelo base sin fine-tuning?

DICES:

Un modelo base solo sabe continuar texto.

Si le escribes «hola, quien eres», te continua una conversacion inventada en vez de contestarte, porque continuar texto es lo unico que le ensenaron.

El mio paso ademas por instruction-tuning, que le ensena que una pregunta se responde.

Por eso el identificador lleva la palabra Instruct.

Sin esa etapa el system prompt no serviria de nada:

un modelo base no obedece instrucciones, las continua.

Las tres etapas en orden: preentrenamiento (aprende a continuar texto), fine-tuning (se especializa en un dominio), instruction-tuning (aprende a obedecer).

¿Que pasa si la conversacion excede la ventana de contexto?

Respondida en el paso 3. Anade esto si insisten:

DICES:

Elena no se rompe: simplemente deja de acordarse del principio de la conversacion.

Y el contador avisa antes de que pase.

Las seis, en una linea cada una

- Token → el pedazo mas chico que el modelo entiende. Ve numeros, no letras.
- Self-attention → cada palabra decide a quien mirar. Da el significado en contexto.
- Ventana de contexto → no recuerda; se le reenvia todo. Guardo diez turnos.
- Personalidad → la define el system prompt, y nada mas.
- Base vs Instruct → el base continua, el Instruct responde.
- Temperatura → cuanto riesgo corre al elegir la palabra siguiente.

PARTE 4

Otras preguntas que pueden caer

Solo si preguntan. Es tu red de seguridad.

¿Donde esta el softmax en tu codigo?

DICES:

No hay ningun softmax en mi codigo, y es correcto que no lo haya:

el softmax ocurre dentro del modelo, y yo uso un modelo preentrenado.

Lo que si hago es comprobarlo. Cojo una fila real de la matriz de atencion,

la sumo, y verifico que da exactamente uno. Si no lo diera, el programa se detiene.

Suma uno porque es una distribucion de probabilidad, que es lo que produce un softmax.

Y ademas anadi una demostracion: aplico un softmax sobre tres puntajes cualesquiera

y se ve la operacion entera, entran numeros sueltos y salen probabilidades que suman uno.

Linea 424. Ahi mismo esta la tabla que demuestra que la temperatura es ese mismo calculo dividiendo los puntajes antes.

¿Que son [CLS] y [SEP]?

DICES:

Los anade el tokenizador, no los escribo yo.

El [CLS] va al principio y viene de classification: es una casilla que recoge

un resumen de toda la frase, y se usa cuando la tarea es clasificar.

El [SEP] marca el final, y separa las frases cuando le das dos a la vez.
En mi laboratorio no los uso para nada, porque no estoy clasificando:
solo quiero los embeddings y la atencion. Aparecen porque el modelo siempre los pone.
Por eso mi frase de dieciocho pedazos da veinte tokens: los dos de mas son estos.

¿Y por que no usas el [CLS]?

DICES:

Porque mi objetivo era enseñar el mecanismo, no clasificar.
Para eso necesito un vector por token, y el [CLS] hace lo contrario:
aplasta la frase entera en un solo vector.
Ademas, un [CLS] sin fine-tuning es un mal resumen de la frase, es un resultado conocido.
Para que sirviera tendria que ajustar el modelo, y el enunciado dice que no se espera
que entrenemos nada.

¿Para que sirve el mapa de atencion?

DICES:

No es una ilustracion que hice aparte: son los numeros reales que el modelo calcula.
Cada fila dice en que proporcion se mezclan las demas palabras
para construir el significado de esa.
Si borrara el codigo que dibuja el mapa, el asistente funcionaria igual:
esos numeros se seguirian calculando. Lo unico que perderia es poder verlos.

¿Por que no puedes ver los pesos del modelo de la nube?

DICES:

Porque ese modelo no corre en mi computadora: corre en los servidores del proveedor.
Lo unico que hay entre mi programa y el es una direccion web que recibe texto
y devuelve texto. Los pesos de atencion existen dentro de su servidor
mientras calcula, y ahi se quedan. No hay ningun parametro para pedirlos.
Por eso el laboratorio usa BETO, que se descarga y corre dentro de mi programa.

Matiz importante: NO es que ese modelo no tenga atencion. Todos los Transformers la tienen. Es que no te deja verla.

¿Que modelo usa cada cosa, y por que ese?

Whisper large-v3 (OpenAI) oye. Entrenado en muchos idiomas: el espanol le sale bien.
Qwen3.8 (Alibaba) piensa. Es de razonamiento: trabaja el problema antes de contestar.
Llama-3.3-70B (Meta) es el alterno. 70B son setenta mil millones de parametros.
edge-tts habla. No es un modelo: es una libreria que usa el servicio de voz de Microsoft.
BETO (Universidad de Chile) no conversa. 110 millones de parametros, solo espanol, y se deja abrir por dentro.

¿Por que dos modelos de lenguaje y no uno?

DICES:

Por una incidencia real. A tres dias de la entrega el proveedor retiro el alternativo
que tenia puesto. Probe los ciento sesenta y nueve modelos del catalogo
y solo veinte respondian.
Los elegi de familias distintas a proposito: si los dos vienen de la misma,
un problema del proveedor se los lleva a la vez.

¿Como funciona sin internet?

DICES:

Si se cae la red, las tres piezas cambian solas a equivalentes que corren en mi laptop.

Responde bastante peor, y no tiene sentido disimularlo: el modelo local es unas mil veces mas pequeno. A cambio, responde. Sin eso, quedarse sin red dejaba la aplicacion muerta.

Y el cambio no es ciego: distingue entre quedarse sin red y tener la clave mal escrita.

Si es la clave, no cambia a local, porque eso taparia un error de configuracion.

Como lo hice sin duplicar codigo: envolvi las tres piezas de nube, cada una con su equivalente local. El orquestador ni se entera de que existen.

¿Que fue lo mas dificil?

DICES:

Un fallo silencioso en el laboratorio. Pedia los pesos de atencion y me llegaban vacios, sin lanzar ninguna excepcion.

La implementacion de atencion que el modelo carga por defecto no llega a construir esos pesos, porque es mas rapida asi, y hay que pedirle la clasica a mano.

Lo peor era el silencio: un fallo que no se queja se te cuela hasta el final.

Ahora el codigo comprueba que los pesos existan y se detiene si vuelven vacios.

¿Es seguro que el modelo ejecute cosas en tu computadora?

DICES:

Todo lo que toca el sistema pasa antes por una lista blanca.

La calculadora, por ejemplo, no usa la funcion de evaluar de Python, que correria cualquier cosa que al modelo se le ocurra escribir: analiza la expresion y admite aritmetica y nada mas.

Las paginas y las carpetas que se pueden abrir salen de una lista cerrada.

El modelo propone la accion, pero decidir que se ejecuta le corresponde al programa.

¿Cuántas pruebas tienes?

DICES:

Ciento ochenta y nueve pruebas automaticas, que corren en unos doce segundos sin red, sin microfono y sin gastar saldo de la API.

Si quieren, las ejecuto ahora.

PARTE 5

Limitaciones y etica

Las limitaciones las sacas TU, en medio minuto. La etica, solo si preguntan.

Dilas tu primero. Que te las saquen ellos, resta.

Limitaciones

DICES:

La mas seria es que el modelo puede alucinar: inventarse un dato y decirlo con el mismo tono de seguridad que uno cierto.

Eso no se arregla con codigo, porque no es un fallo del programa sino de como funciona un modelo de lenguaje. Lo que si hice fue avisarlo.

Sin internet responde bastante peor.

El modelo principal es cerrado: no puedo ver sus pesos ni saber con que datos lo entrenaron, así que tampoco puedo auditar sus sesgos.
La transcripción falla con ruido de fondo o si se habla lejos del microfono.
Y depende de que quede saldo en la API.

Lo de no poder auditar los sesgos es un nivel mas: reconoces el limite de tu propia revision, no solo el del sistema.

Etica

- Ninguna clave en el repositorio. Van en .env, que git ignora.
- Lo compruebo sobre todo el historial, no solo el ultimo commit.
- Elena avisa al arrancar que es una IA y que puede equivocarse.
- Tambien lo lleva en su system prompt: si le preguntan que es, lo responde.
- La voz no se guarda en disco. Se transcribe y se descarta.
- Limitaciones documentadas en el informe, no escondidas.
- Modelos y proveedores citados explicitamente.

PARTE 6

Si algo sale mal, y el cierre

Lo de arriba solo si falla algo. El cierre SIEMPRE.

Si algo falla en vivo

Regla unica: no te calles. Narra lo que pasa.

Tarda mucho → "esto es latencia de red: la peticion viaja a un servidor y vuelve"

No entiende → repite mas cerca del micro. Explica que fallo el reconocimiento de voz, no el modelo.

Cae la red → perfecto: ensena el modo sin internet. Suma en vez de restar.

Sale un error → "esto es el manejo de errores funcionando: la aplicacion no se cae"

Si no sabes algo

No inventes. Es lo unico que puede hundirte de verdad.

DICES:

Eso no lo medi. Lo que si compruebo es esto otro...

Y lo llevas a algo que si hiciste. Reconocer un limite y redirigir a evidencia real demuestra que sabes distinguir lo que verificaste de lo que no.

El cierre

DICES:

En resumen: el pipeline completo funciona de extremo a extremo con voz real, el laboratorio evidencia los cuatro conceptos con la salida del propio programa, hay ciento ochenta y nueve pruebas automaticas, ninguna credencial esta en el repositorio, y el asistente sigue funcionando sin internet, que no lo pedian.
Lo que no puedo arreglar con codigo, que es que el modelo se equivoque, esta documentado y avisado.